

DOCUMENTACIÓN DEL PROYECTO: COMPRESOR DE TEXTO HUFFMAN

Geor Sebastián Gómez Correa
Tomás Camilo García López
Eduardo Castellanos Márquez



1. GUÍA DE USUARIO

Esta sección explica cómo ejecutar el programa y qué esperar durante su funcionamiento.

Requisitos Previos

- Tener instalado **Python 3.8** o superior.
- No se requieren librerías externas (solo librerías estándar de Python).

Pasos para Ejecutar

1. Preparación del Archivo:

- El programa buscará un archivo llamado `texto_huffman.txt` en la misma carpeta del proyecto.
- Si no existe, el programa creará uno automáticamente con un mensaje de prueba.
- **Recomendación:** Abre `texto_huffman.txt` y pega allí el texto largo (ej. libro, artículo) que desees comprimir.

2. Ejecución:

- Abre tu terminal o consola en la carpeta del proyecto.
- Ejecuta el comando: `python main.py`

3. Resultados en Consola:

- El programa mostrará el progreso paso a paso: Lectura → Compresión → Guardado → Descompresión → Verificación.
- Verás una tabla comparativa de tamaños (Bytes originales vs. Bytes comprimidos) y el porcentaje de reducción.
- Verás una tabla de "Verificación Semántica" comparando las palabras más frecuentes.

4. Archivos Generados:

- `texto_comp.bin`: El archivo comprimido en formato binario.
- `texto_descomp.txt`: El archivo recuperado tras la descompresión.

2. DOCUMENTO DE ESTRUCTURAS DE DATOS Y COSTOS

Análisis profundo de las estructuras seleccionadas, su implementación y justificación de rendimiento.

A. Diccionarios (Hash Maps)

- **Dónde se usan:**
 - Conteo de frecuencias de caracteres (`collections.Counter`).
 - Mapa de Códigos Huffman (Carácter → Bits).
 - Diccionario Inverso para descompresión (Bits → Carácter).
 - Implementación de la Matriz Dispersa (DOK).
- **Justificación:** Necesitamos acceso instantáneo a los valores asociados a una clave (ya sea un carácter, un código binario o una coordenada de matriz).
- **Complejidad Temporal:**
 - Inserción/Búsqueda: $O(1)$ (promedio).
- **Costo en Memoria:** Medio-Alto. Python reserva más espacio del necesario para evitar colisiones de hash, pero la velocidad justifica el costo.

B. Cola de Prioridad (Min-Heap)

- **Dónde se usa:** Construcción del Árbol de Huffman ([heapq](#)).
- **Justificación:** El algoritmo de Huffman es "voraz" (greedy); necesita acceder repetidamente a los dos nodos con *menor* frecuencia. Un Min-Heap mantiene el elemento menor en la raíz siempre disponible.
- **Complejidad Temporal:**
 - Insertar nodo: $O(\log K)$.
 - Extraer mínimo: $O(\log K)$.
 - (Donde K es el número de caracteres únicos).
- **Costo en Memoria:** Muy Bajo. Se implementa sobre una lista simple (array), sin necesidad de punteros complejos.

C. Árbol Binario (Huffman Tree)

- **Dónde se usa:** Representación lógica de la codificación de prefijos.
- **Justificación:** Es la estructura natural para generar códigos de longitud variable donde ningún código es prefijo de otro. Los nodos hoja contienen los caracteres y los nodos internos las sumas de frecuencias.
- **Complejidad Temporal:**
 - Construcción total del árbol: $O(K \log K)$.
- **Costo en Memoria:** Medio. Se crean objetos [NodoHuffman](#) en el *heap* de memoria, cada uno con referencias a sus hijos.

D. Matriz Dispersa (DOK - Dictionary of Keys)

- **Dónde se usa:** Verificación semántica y análisis de vocabulario ([sparse_matrix_analyzer.py](#)).
 - **Justificación:** Un texto largo tiene un vocabulario amplio, pero cada palabra aparece pocas veces. Una matriz tradicional (densa) estaría llena de ceros (espacio desperdiciado). La estructura DOK solo almacena los valores no nulos.
 - **Implementación:** Un diccionario donde la clave es una tupla [\(0, id_palabra\)](#) y el valor es la frecuencia.
 - **Complejidad Temporal:**
 - Acceso a celda: **$O(1)$** .
 - **Costo en Memoria:** Óptimo para datos dispersos ($O(\text{NNZ})$): proporcional al número de elementos no cero).
-

3. DOCUMENTACIÓN TÉCNICA (Funciones y Librerías)

Detalle de la arquitectura del software.

Librerías Estándar Utilizadas

- **heapq**: Implementación eficiente del algoritmo de cola de prioridad (heap) para construir el árbol.
- **collections** (**Counter**, **defaultdict**): Estructuras de alto rendimiento para conteo y diccionarios con valores por defecto.
- **pickle**: Serialización de objetos Python. Vital para guardar la metadata (el mapa de códigos) dentro del archivo binario **.bin**.
- **os**: Operaciones del sistema operativo para verificar existencia y tamaño de archivos.

Módulo: **huffman_compressor.py**

Encargado de la lógica de compresión y manejo binario.

- **Clase **NodoHuffman**:**
 - Define la estructura del nodo (carácter, frecuencia, hijos).
 - Sobrecarga el operador **__lt__** para permitir la comparación automática en el Heap.
- **construir_arbol(texto):**
 - Crea el mapa de frecuencias y utiliza el Min-Heap para fusionar nodos hasta obtener la raíz del árbol.
- **_cadena_a_bytes(bits_cadena):**
 - **Función crítica de bajo nivel.** Convierte la cadena de texto "0101..." en enteros de 8 bits (Bytes) reales. Maneja el *padding* (relleno) si la longitud no es múltiplo de 8.
- **guardar_binario(...):**
 - Persistencia de datos. Usa **pickle** para guardar la cabecera (padding + mapa de códigos) y luego escribe los bytes crudos.

- **descomprimir_mapa(...):**
 - Reconstruye el texto utilizando el diccionario de códigos invertido. Itera sobre los bits y acumula hasta encontrar una coincidencia.

Módulo: **sparse_matrix_analyzer.py**

Encargado del análisis semántico y verificación.

- **Clase **AnalizadorTextoHuffman**:**
 - Gestiona el vocabulario único del texto.
- **_tokenizar_simple(texto):**
 - Normaliza el texto: convierte a minúsculas, elimina caracteres Unicode problemáticos (para evitar errores de consola) y limpia la puntuación básica, respetando tildes y eñes.
- **construir_matriz_dispersa_dok(...):**
 - Genera la estructura de datos dispersa mapeando (**Documento, Palabra**) -> **Frecuencia**.
- **obtener_top_frecuencias(...):**
 - Extrae, ordena y devuelve las \$N\$ palabras más frecuentes para la comparación.

Módulo: **main.py**

Controlador principal.

- **demo_huffman():**
 - Orquesta el ciclo completo: Lectura -> Compresión -> Escritura Binaria -> Lectura Binaria -> Descompresión -> Escritura Texto.
- **comparar_frecuencias_semanticas():**
 - Ejecuta el analizador de matriz dispersa sobre el archivo original y el descomprimido, imprimiendo una tabla comparativa para validar la integridad de los datos.